

1. โปรแกรมข้างล่างนี้มีที่ผิดทั้งหมด 10 คำสั่ง จงหาว่ามีคำสั่งใดบ้างที่ผิด ห้ามตอบเกิน 10 คำสั่ง

```
#include<iostream>
using namespace std;
class A {
    int a;
    protected:
    int f1(int i) { return a=i; }
    public:
    A() { a=1; }
    A(int x) { a=x; }
    A f2(int i) { a=i; return a; }
    int f3() { return a; }
    A f4(A &x) { x.a = ++a; return a; }
    int f5(A x, int i) { return f2(i)+x.f3(); }
    A f6() { a*=a; return *this; }
    operator int() { return a; }
};
class B: public A {
    int b;
    protected:
    int g1(int i) { b=f1(i); return *this; }
    public:
    B() { b=1; }
    B(int i) { b=i; }
    B(A x) { b = 1+x.a; }
    B g2(B &y) { y.f1(b); b+=2; return b; }
    B g3(A &x) { x.f1(b); return *this; }
    A g4(int i) { *this=i; return f6()+2; }
    A g5(A &x) { return f4(x).f6(); }
    B g6(A x, int j) { b=f5(x, j); return *this; }
    A g7(B y) { return f2(g1(y.b)); }
};
class C: public B {
    B y;
    protected:
    B h1(A x) { y.g4(x); return y; }
    public:
    C h2(C k) { y.g7(k.y).f3(); return *this; }
    C h3(int k) { y.g4(f6()+g1(k)); return *this; }
    A h4(B &y) { return f2(y.g1(1)); }

    A* h5(int k) { g7(y).f6(); return this; }
    int h6(A &x) { return f1(y.g5(x).f3()+*this); }
    B h7(B k) { return y.g6(*this, f3()); }
    A h8(A x) { return (x+y).f6(); }
    B h9(B k) { return (y>k)?y:k; }
    friend B f(B&, A);
};
ostream& operator<<(ostream &o, C &x) {
    o<<x; return o;
}
B f(B &y, A x) {
    C c;
    y=c.h1(x);
    x=y+x.f6();
    c.h1(x).f3();
    c.y.g1(x.f3());
    return c.y;
}
int main() {
    A a;
    B b;
    C c;
    cout<<c.h9(b).g5(a)<<endl;
    int k;
    k = c.h6(b.g4(a.f3()));
    k = c.g6(b.g4(a.f3()), 5).f3();
    a=5;
    b=6;
    c=7;
    k = a+b+c;
    k = a.f5(c.f3(), b.f3());
    k = b.h7(c.g6(a, 1)).f3();
    k = b.f5(*c.h5(3), a.f2(4));
    k = c.h3(b.g1(a.f3())).f3();
    b = c.h5(4)->f3();
    c = b.g4(a.f3()).f3();
    a = *c.h5(b.f6());
}
```

2. โปรแกรมข้างล่างนี้มีคำตอบทั้งหมด 71 บรรทัด จงแสดงผลลัพธ์ทั้ง 71 บรรทัด

```
#include<iostream>
using namespace std;
class A {
    int a;
public:
    A() { a=0; cout<<"A "<<a<<endl; }
    A(int i) { a=i; cout<<"AA "<<a<<endl; }
    virtual ~A() { cout<<"a= "<<a<<endl; }
    virtual int operator[](int i) { return a+i; }
    int set(int i) { (*this)(i); return a; }
    virtual A* operator()(int i) {
        cout<<"hey A"<<endl;
        a=i;
        return this;
    }
    int get() { return a; }
    operator int() { return a; }
};
```

```
class B: virtual public A {
    int b;
public:
    B() { b=2; cout<<"B "<<b<<endl; }
    ~B() { cout<<"b= "<<b<<endl; }
    int operator[](int i) { return b+i; }
    A* operator()(int i) {
        cout<<"hi B"<<endl;
        b=i;
        return this;
    }
    virtual int get() { return b; }
    operator int() { return b; }
};
```

```
class C: public B {
    int c;
public:
    C() { c=3; cout<<"C "<<c<<endl; }
    ~C() { cout<<"c= "<<c<<endl; }
    int operator[](int i) { return c+i; }
```

```
    A* operator()(int i) {
        cout<<"hello C"<<endl;
        c=i;
        return this;
    }
    int get() { return c; }
    virtual operator int() { return c; }
};
```

```
class D: virtual public A, public C {
    int d;
public:
    D():A(5) { d=4; cout<<"D "<<d<<endl; }
    ~D() { cout<<"d= "<<d<<endl; }
    int operator[](int i) { return d+i; }
    A* operator()(int i) {
        cout<<"SawasD"<<endl;
        d=i;
        return this;
    }
    int get() { return d; }
    operator int() { return d; }
};
```

```
class E: public A {
    int e;
public:
    E() { e=9; cout<<"E "<<e<<endl; }
    ~E() { cout<<"e= "<<e<<endl; }
    int operator[](int i) { return e+i; }
    A* operator()(int i) {
        cout<<"hihi E"<<endl;
        e=i;
        return this;
    }
    int get() { return e; }
    operator int() { return e; }
};
```

```

int main() {
    A *a[3];
    B *b[2];
    C *c = new C;
    D *d = new D[3];
    E e;
    cout<<"about a[i]"<<endl;
    a[0] = c;
    a[1] = &d[0];
    a[2] = &e;
    for (int i=0; i<3; i++) {
        a[i]->set(i+1);
        cout<<*a[i]<<endl;
    }
    cout<<e<<endl;
    cout<<"ee "<<e(7)->get()<<endl;
    cout<<e<<endl;
    cout<<(*c)[3]<<endl;
    cout<<*c<<endl;

    for (int i=0; i<3; i++) {
        cout<<d[i](i+2)->get()<<endl;
        cout<<d[i].get()<<endl;
        cout<<d[i]<<endl;
    }
    delete c;
    c = &d[0];
    b[0] = &d[1];
    b[1] = &d[2];
    cout<<(*c)[2]<<endl;
    cout<<c->get()<<endl;
    cout<<*c<<endl;
    for (int i=0; i<2; i++) {
        cout<<(*b[i])(i+6)->get()<<endl;
        cout<<b[i]->get()<<endl;
        cout<<*b[i]<<endl;
    }
    delete[] d;
    cout<<".....End....."<<endl;
}

```

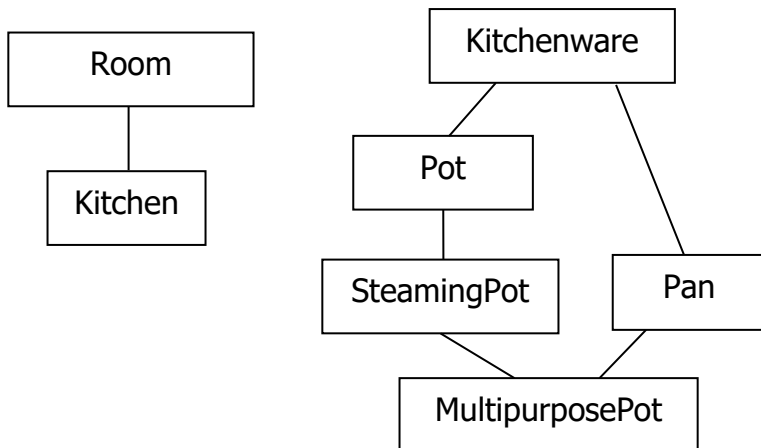
3. โปรแกรมข้างล่างนี้มีคำตอบทั้งหมด 37 บรรทัด จงแสดงผลลัพธ์ทั้ง 37 บรรทัด

```
#include<iostream>
using namespace std;
class A {
    int a;
    public:
    A() {
        a=3;
        cout<<"A"<<endl;
    }
    A(int i) {
        a=i;
        cout<<"good"<<endl;
    }
    ~A() {
        cout<<"bye A "<<*this<<endl;
    }
    A(A &p) {
        cout<<"copy"<<endl;
        a=p.a;
    }
    A operator+=(A &k) {
        cout<<"call+="<<endl;
        a=k+a;
        return *this;
    }
    int operator+(A &k) {
        cout<<"plus+"<<endl;
        return a+k;
    }
    operator int() {
        cout<<"I am int"<<endl;
        return a+1;
    }
    friend ostream& operator<<(ostream& os,
        A &a) {
        os<<"a="<<a.a;
        return os;
    }
};
```

```
class B {
    A *a;
    int n;
    public:
    B(int i) { n=i; a = new A[n]; }
    ~B() {
        cout<<"bye B "<<endl;
        delete[] a;
    }
    A operator()(A x, int i) {
        cout<<"operator(A) "<<endl;
        a[i]=x;
        return a[i];
    }
    void operator++() {
        for (int i=0; i<n; i++)
            a[i]=a[i]+1;
    }
    int operator[](int i) {
        cout<<"["<<endl;
        return a[i];
    }
    friend ostream& operator<<(ostream& os,
        B &b) {
        for (int i=0; i<b.n; i++)
            cout<<b[i]<<endl;
        return os;
    }
};

int main() {
    A a1, a2(1);
    cout<<a1+a2<<endl;
    a1+=a2;
    cout<<a1<<endl;
    B b(2);
    cout<<b(a1,0)<<endl;
    cout<<b;
    ++b;
    cout<<"-- OOP --"<<endl;
}
```

จงสร้างคลาสตามรูปด้านล่าง โดยมีการรับทอดตามรูปและไม่เกิดความกำกวม โดยกำหนดให้การรับทอดทั้งหมดเป็นการรับทอดแบบ public ทุกคลาสให้เก็บแอทริบิวต์ทั้งหมดไว้ในส่วนของ private และกำหนดให้ Kitchenware เป็น Abstract class โดยให้สร้าง void increasePrice() และ string getDetail() เป็น pure virtual function



แต่ละคลาสประกอบด้วยแอทริบิวต์และความหมาย ดังนี้

	แอทริบิวต์	ความหมาย
Kitchenware	double price; string brand;	ราคา ยี่ห้อ
Pot	int typePot;	1.หม้อสองหู 2.หม้อด้ามจับยาว
SteamingPot	int layer;	จำนวนชั้นของหม้อหนึ่ง 1, 2 หรือ 3 ชั้น
Pan	int purpose;	1.กระทะใช้ทั่วไป 2.กระทะสำหรับย่าง
MultipurposePot	bool shabu;	true: ทำชาบู 2 น้ำซุ๊ปได้ false: ทำ 2 น้ำซุ๊ปไม่ได้
Room	int style; bool hasAir;	1.โมเดิร์น 2.มินิมอล 3.คลาสสิก true: มีแอร์ false: ไม่มีแอร์
Kitchen	int type; Kitchenware **k; int num;	1.ครัวไทย 2.ครัวฝรั่ง อาร์เรย์เก็บพอยเตอร์ที่ชี้ไปยัง Kitchenware ขนาดของอาร์เรย์ k

จงสร้างฟังก์ชันต่างๆ ในส่วนของ public ดังนี้

ทุกคลาส จงสร้าง

- default constructor ในการกำหนดค่าให้กับแอทริบิวต์ทุกตัว โดยมีการกำหนดค่าเริ่มต้นดังนี้
 - Kitchenware ราคา 350 ยี่ห้อ "AA"
 - Pot หม้อสองหู ราคา 350 ยี่ห้อ "AA"
 - SteamingPot หม้อสองหู จำนวนชั้น 1 ชั้น ราคา 450 ยี่ห้อ "AA"
 - Pan กระทะใช้ทั่วไป ราคา 250 ยี่ห้อ "AA"
 - MultipurposePot หม้อด้ามจับยาว มีจำนวนชั้น 1 ชั้น มีกระทะสำหรับย่าง ทำชาบู 2 น้ำซุ๊ปไม่ได้ ราคา 650 ยี่ห้อ "AA"
 - Room ห้องสไตล์คลาสสิก ไม่มีแอร์
 - Kitchen ห้องสไตล์คลาสสิก ครัวไทย ไม่มีแอร์ num=0 k=0

- ฟังก์ชัน set ในการกำหนดค่าให้กับแอทริบิวต์แต่ละตัว
 - o กรณี price ค่าต้องมากกว่าหรือเท่ากับศูนย์
 - o กรณี layer มีค่าแค่ 1, 2 หรือ 3
 - o กรณี k ให้สร้างฟังก์ชัน setK(int, Kitchenware*) ในการกำหนดค่าให้กับ k โดยให้กำหนดค่าในอาร์เรย์ที่ละ 1 ค่า โดยระบุตำแหน่งที่เก็บในอาร์เรย์
 - o กรณี num ค่าต้องมากกว่าหรือเท่ากับศูนย์ และให้คืนพื้นที่เดิมและจองพื้นที่ใหม่ให้กับ k ด้วย
 - o กรณี typePot, purpose, style, type ให้กำหนดค่าให้อยู่ในขอบเขตที่กำหนดตามคำอธิบาย
- ฟังก์ชัน get ในการรีเทิร์นแอทริบิวต์แต่ละค่า เช่น getPrice()
 - o สำหรับ k ให้รีเทิร์นค่าสมาชิกในอาร์เรย์ที่ละ 1 ค่าโดยระบุตำแหน่งที่เก็บในอาร์เรย์

คลาส Kitchenware, Pot, SteamingPot, Pan, MultipurposePot จงสร้าง

- constructor โดยรับข้อมูลจากพารามิเตอร์ในการกำหนดค่าให้กับแอทริบิวต์ทุกตัวรวมทั้งbase class โดยใช้วิธีการไปเรียกใช้ constructor ของ base class และให้ใช้เงื่อนไขเดียวกันกับฟังก์ชัน set
- string getDetail() โดยที่คลาส Kitchenware ให้สร้าง เป็น pure virtual function ในคลาสอื่นๆ ให้รีเทิร์นค่าดังนี้
 - o Pot ให้รีเทิร์นคำอธิบาย typePot เช่น "two-handled pot"
 - o SteamingPot ให้รีเทิร์นคำอธิบาย เช่น "steaming pot one layer"
 - o Pan ให้รีเทิร์นคำอธิบาย purpose เช่น "grill pan"
 - o MultipurposePot ให้รีเทิร์นคำอธิบาย เช่น "cannot make 2 pots of shabu"
- ให้สร้างฟังก์ชันในการ cast อ็อบเจกต์ของ Pot, SteamingPot, Pan และ MultipurposePot ไปเป็น string โดยใช้หลักการ dynamic binding ที่ใช้ getDetail() ในการแปลง
- void increasePrice() โดยที่คลาส Kitchenware ให้สร้าง เป็น pure virtual function ในคลาส Pot, SteamingPot, Pan และ MultipurposePot ให้เพิ่มค่า price ตามลำดับดังนี้ 10, 15, 20, 25 เช่น ถ้าเดิมหม้อหนึ่งราคา 350 ก็จะถูกเพิ่มเป็น 365 เป็นต้น

คลาส MultipurposePot จงสร้าง

- ฟังก์ชันในการกำหนดค่าให้กับแอทริบิวต์ต่างๆ อย่างถูกต้องและมีความหมาย โดยใช้หลักการ function overloading ทั้งหมด 3 ฟังก์ชัน

คลาส Kitchenware จงสร้าง

- ฟังก์ชันในการนับจำนวนอ็อบเจกต์ที่เป็น Kitchenware ว่า ณ ปัจจุบันมีจำนวนอ็อบเจกต์คงอยู่ในโปรแกรมทั้งหมดเท่าไร ถึงแม้ว่าจะไม่มีอ็อบเจกต์อยู่เลยก็ต้องสามารถบอกได้ด้วย ให้สร้างแอทริบิวต์และฟังก์ชันที่เกี่ยวข้องเพิ่มเติมได้

คลาส Room, Kitchen จงสร้าง

- ฟังก์ชัน show() โดยให้แสดงข้อมูลทุกอย่างของคลาสตนเองและของ superclass ออกทางจอภาพสำหรับสมาชิกใน k ให้ใช้ getDetail() ในการดึงข้อมูล และให้ทำ show() เป็น virtual function
- operator << ในการเรียกใช้ show() โดยใช้หลักการ dynamic binding

คลาส Kitchen จงสร้าง

- destructor ในการคืนพื้นที่ของแอทริบิวต์ที่มีการจอง
- copy constructor ในการก๊อปปี้ค่าทุกอย่างถูกต้อง และ k มีพื้นที่เป็นของตัวเอง
- double getPriceBrand(string) ให้รับชื่อของ brand และคำนวณราคาเครื่องครัวของ brand นี้ทั้งหมด แล้วรีเทิร์นค่าผลรวมออกจากฟังก์ชัน แต่ถ้าไม่พบ brand นี้ให้รีเทิร์นค่าศูนย์

ใน main

- จงสร้าง
 - o pot เป็นอ็อบเจกต์ของ Pot ให้สร้างโดยไปเรียกใช้ default constructor และกำหนดค่าเป็น หม้อต้มจืด ราคา 380 ยี่ห้อ "BB"
 - o sPot เป็นอาร์เรย์ขนาด 2 เก็บอ็อบเจกต์ของ SteamingPot ให้สร้างโดยไปเรียกใช้ default constructor และให้กำหนดค่าดังนี้
 - sPot[0] เป็นหม้อสองหู จำนวนชั้น 1 ชั้น ราคา 550 ยี่ห้อ "CC"
 - sPot[1] เป็นหม้อสองหู จำนวนชั้น 2 ชั้น ราคา 720 ยี่ห้อ "BB"จากนั้นให้แสดงรายละเอียดทั้งหมดของ pot และ Spot ออกทางจอภาพ
- จงสร้าง
 - o pan เป็นพอยเตอร์ที่ชี้ไปยังคลาส Pan ให้จองพื้นที่เป็นอาร์เรย์ขนาด 3 เก็บอ็อบเจกต์ของ Pan โดยให้ไปเรียกใช้ default constructor และกำหนดค่าดังนี้
 - pan[0] กระทะใช้หัวไป ราคา 290 ยี่ห้อ "AA"
 - pan[1] กระทะย่าง ราคา 790 ยี่ห้อ "DD"
 - pan[2] กระทะใช้หัวไป ราคา 390 ยี่ห้อ "BB"
 - o mPot เป็นพอยเตอร์ที่ชี้ไปยังคลาส MultipurposePot จากนั้นให้จองพื้นที่เพื่อสร้าง อ็อบเจกต์โดยให้ไปเรียกใช้ constructor ที่รับพารามิเตอร์ โดยกำหนดค่าเป็น หม้อสองหู มีจำนวนชั้น 1 ชั้น มีกระทะสำหรับใช้หัวไป ทำชาบู 2 น้ำซุปรด ราคา 1650 ยี่ห้อ "CC"จากนั้นให้แสดงรายละเอียดทั้งหมดของ pan และ mPot ออกทางจอภาพ
- จงแสดงค่าของ pan ออกทางจอภาพโดยเรียกใช้ cast operator ที่ นศ สร้างไว้
- จงสร้าง room1, room2, room3, room4 เป็นพอยเตอร์ที่ชี้ไปยัง Room, Kitchen, Kitchen, Room ตามลำดับ ให้จองพื้นที่และกำหนดค่าต่างๆ ดังนี้
 - o room1 เป็นห้องสไตล์คลาสสิก ไม่มีแอร์
 - o room2 เป็นห้องครัว สไตล์มินิมอล ครัวฝรั่ง มีแอร์ มีเครื่องครัว pot และ mPot
 - o room3 เป็นห้องครัว สไตล์โมเดิร์น ครัวไทย ไม่มีแอร์ มีเครื่องครัว pan[0] และ sPot[0]
 - o room4 เป็นห้องสไตล์โมเดิร์น มีแอร์จากนั้นให้สร้าง restaurant เป็นอาร์เรย์ขนาด 4 เก็บพอยเตอร์ที่ชี้ไปยัง Room โดยกำหนดให้สมาชิกแต่ละตัวชี้ไปที่ room1, room2, room3, room4 ตามลำดับ และแสดงค่าใน restaurant ทั้งหมดออกทางจอภาพโดยใช้ operator<< ที่ นศ สร้างไว้
- จงสร้าง onlyKitchen เป็นพอยเตอร์ที่ชี้ไปยังอ็อบเจกต์ของ Kitchen โดยจองพื้นที่เป็นห้องครัว สไตล์มินิมอล ครัวไทย ไม่มีแอร์ มีเครื่องครัว pan[1], pan[2] และ sPot[1] และ จงสร้าง house เป็นอาร์เรย์ขนาด 4 เก็บพอยเตอร์ที่ชี้ไปยัง Room โดยให้
 - o house[0] ชี้ไปที่ room3
 - o house[1] จองพื้นที่เป็นห้องสไตล์โมเดิร์น ไม่มีแอร์
 - o house[2] จองพื้นที่เป็นห้องสไตล์คลาสสิก มีแอร์
 - o house[3] ชี้ไปที่ onlyKitchenและ จากนั้นให้แสดงค่าใน house ทั้งหมดออกทางจอภาพโดยให้เรียกใช้ show

- จงสร้าง copyKitchen เป็นพอยเตอร์ที่ชี้ไปยังคลาส Kitchen จากนั้นให้จองพื้นที่เพื่อสร้าง อ็อบเจกต์โดยให้ไปเรียกใช้ copy constructor ในการก๊อปปี้ค่ามาจาก onlyKitchen จากนั้นให้แสดงค่าของ copyKitchen ออกทางจอภาพ
- จงสร้างอ็อบเจกต์และเรียกใช้ operator overloading ที่ นศ สร้างไว้ใน MultipurposePot ทั้ง 3 ฟังก์ชัน และแสดงค่าทุกค่าของอ็อบเจกต์ออกทางจอภาพ
- จงสร้าง Room **homeKitchen[2]; โดยที่สมาชิกใน homeKitchen แต่ละตัวเก็บพอยเตอร์ที่ชี้ไปยัง house, restaurant ตามลำดับ จากนั้นให้หาว่าใน homeKitchen มีห้องสไตล์โมเดิร์น มินิมอล และคลาสสิกอยู่อย่างละกี่ห้อง ให้แสดงผลลัพธ์ออกทางจอภาพ
- จงสร้างฟังก์ชัน virtual bool isKitchen() เพิ่มเติมใน Room และ Kitchen โดยที่ใน Room ให้รีเทิร์น false เสมอ และใน Kitchen ให้รีเทิร์น true เสมอเพื่อที่จะใช้ในการตอบคำถามข้อนี้และข้อถัดไป จากนั้นให้เรียกใช้ getPriceBrand(string) คำนวณผลรวมของราคาเครื่องครัวยี่ห้อ BB ทั้งหมดที่อยู่ใน homeKitchen และแสดงผลลัพธ์ออกทางจอภาพ
- จงแสดงเฉพาะรายละเอียดของเครื่องครัวทั้งหมดที่อยู่ใน homeKitchen ออกทางจอภาพ โดยการเรียกใช้ cast operator ที่ นศ สร้างไว้
- จงคืนพื้นที่ในลำดับที่ถูกต้อง